

邏輯系統

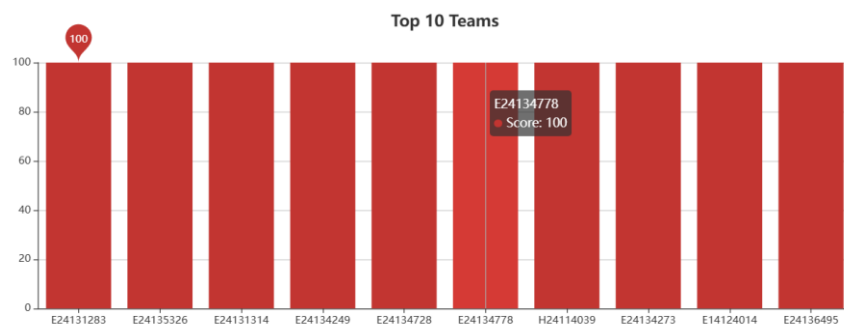
HW2

```

=====
Next Column
=====
Prime Implicant Chart
=====
      | 0 2 5 6 7 8 10 12 13 14 15
-----+-----
0000--10 |   x   x       x       x
0000-0-0 |  x x       x x
0000-1-1 |       x   x       x   x
0000-11- |       x x       x x
00001--0 |           x x x   x
000011-- |           x x x x
    
```

Table 6-1. Determination of Prime Implicants

	Column I			Column II			Column III		
group 0	0	0000	✓	0, 1	000-	✓	0, 1, 8, 9	-00-	
group 1	1	0001	✓	0, 2	00-0	✓	0, 2, 8, 10	-0-0	
	2	0010	✓	0, 8	-000	✓	0, 8, 1, 9	-00-	
	8	1000	✓	1, 5	0-01		0, 8, 2, 10	-0-0	
group 2	5	0101	✓	1, 9	-001	✓	2, 6, 10, 14	--10	
	6	0110	✓	2, 6	0-10	✓	2, 10, 6, 14	--10	
	9	1001	✓	2, 10	-010	✓			
	10	1010	✓	8, 9	100-	✓			
group 3	7	0111	✓	8, 10	10-0	✓			
	14	1110	✓	5, 7	01-1				
				6, 7	011-				
				6, 14	-110	✓			
				10, 14	1-10	✓			



姓名：彭靖凱

學號：E24134778

授課教授：陳中和 教授

一、作業目標

本次作業的目的是

實作 Quine-McCluskey (QM) 方法，藉由程式設計實現布林代數的系統化簡化流程。

撰寫一個程式：

能接受 8 到 10 個變數的輸入，並處理給定的 minterms 與 don't care 項。

根據 Quine-McCluskey 演算法，自動找出：

1. 所有 Prime Implicants (主項)
2. 所有合法的最小 SOP (Sum of Products)
3. 若設定 verbose = true，則完整輸出每一階段的合併步驟 (每一欄)、Prime Implicant Chart，以及涵蓋關係。

二、程式說明

1. 主要結構與函式

本程式實作於 QuineMcCluskey.cpp 中，核心功能封裝於 solveQm() 函式中，輸出所有 Prime Implicants 及所有合法的最小 SOP (Sum of Products) 結果。

```
struct QmSolution {  
    int numVar;  
    vector<string> primes; // 所有 Prime Implicants  
    vector<vector<string>> minimumSops; // 所有最小 SOP  
};
```

```
QmSolution* solveQm(int numVar, vector<int> minterms, vector<int> dontcares,  
bool verbose)
```

參數說明：

1. numVar：變數數量 (8~10)
2. minterms：邏輯函數為 1 的項目
3. dontcares：可以忽略的項目
4. verbose：若為 true，會印出詳細步驟

2. 合併與化簡流程

1. 初始分組：根據 1 的個數將 minterms 與 don't cares 分組。
2. 逐欄合併：使用 canCombine() 將不同欄位的 term 嘗試合併，形成更簡的形式。
3. 標記未合併項為 Prime Implicants。
4. Prime Implicant Chart 建構：列出每個 PI 覆蓋哪些 minterm。
5. Petrick's Method：

- 建立 minterm $\rightarrow$ PI 的映射。
- 使用 DFS 遞迴，列舉所有覆蓋解並選出最小長度者。

結果輸出與測試

若 verbose = true，會輸出：

- 初始分組表
- 每一欄合併結果
- Prime Implicant Chart
- 所有 Prime Implicants 與最小 SOP 組合

Column 1：初始群組

以 1 的數量分組

所有 minterms 與 don't cares

其中：

v 表示 minterm

d 表示 don't care

00000010 是變數位元表示

冒號後是該項的十進位 index

初始分組表（按 1 的個數 Group 排序）

Group	十進位值	二進位表示 (8-bit)
0	0	00000000
1	2	00000010
1	8	00001000
2	5	00000101
2	6	00000110
2	10	00001010
2	12	00001100
3	7	00000111
3	13	00001101
3	14	00001110
4	15	00001111
8 (DC)	255	11111111

```

=====
Column 1
=====
v 00000000: 0
v 00000010: 2
v 00001000: 8
v 00000101: 5
v 00000110: 6
v 00001010: 10
v 00001100: 12
v 00000111: 7
v 00001101: 13
v 00001110: 14
v 00001111: 15
d 11111111: 255
=====

```

第一次合併

圖片展示的是 **Quine-McCluskey 方法** 中的「**第一次合併結果**」，屬於 Column II（第二欄）。這部分是演算法的核心步驟，目的是將能合併的 minterm 組合在一起，形成更簡的布林項（減少一個變數）。

第一次合併（Column II）

本欄顯示哪些 minterm 成功合併，以及其合併後的結果。

在程式中，我們遍歷分組中 group i 與 group i+1 的 term，找出只差一位的 pair，並合併為 -（don't care）形式。例如：

- 0 與 2（00000000 和 00000010）合併為 000000-0；
- 2 與 6（00000010 和 00000110）合併為 00000-10

這個步驟會繼續向下一輪傳遞，直到無法再合併為止。

說明符號：

- - 代表該位元可忽略（已被簡化）
- 右邊的數字為原始 minterm index
- 合併後的位元表示即為下一欄的 term，會進入下一輪合併

第一次合併表（含 don't care 位元 -）

- 每個位元字串長度固定（這裡是 8 位），代表變數 a~h 的值。
- - 表示這個位元位置不同，因此「不重要」或「可忽略」。
- 冒號右側列出所有被合併的原始 minterm。

原始 minterms	原始二進位	合併結果（含 -）
0, 2	00000000, 000000 1 0	000000-0
0, 8	00000000, 0000 1 000	0000-000
2, 6	00000010, 00000 1 10	00000-10
2, 10	00000010, 0000 1 010	0000-010
8, 10	00001000, 000010 1 0	000010-0
8, 12	00001000, 00001 1 00	00001-00
5, 7	00000101, 00000 1 11	000001-1
5, 13	00000101, 0000 1 101	0000-101
6, 7	00000110, 00000 1 1 1	0000011-
6, 14	00000110, 0000 1 110	0000-110
10, 14	00001010, 0000 1 110	00001-10
12, 13	00001100, 0000110 1	0000110-
12, 14	00001100, 00001 1 10	000011-0
7, 15	00000111, 0000 1 111	0000-111
13, 15	00001101, 00001 1 11	000011-1
14, 15	00001110, 0000111 1	0000111-

合併後的位元>: <被合併的原始 minterms>

```
Next Column
=====
000000-0: 0 2
0000-000: 0 8
00000-10: 2 6
0000-010: 2 10
000010-0: 8 10
00001-00: 8 12
000001-1: 5 7
0000-101: 5 13
0000011-: 6 7
0000-110: 6 14
00001-10: 10 14
0000110-: 12 13
000011-0: 12 14
0000-111: 7 15
000011-1: 13 15
0000111-: 14 15
```

第二次合併 (Column III)

說明原本的：

0 2 → 000000-0

8 10 → 000010-0

兩者合併後變為 0000-0-0，也就是這組能表示 4 個 minterms。這些結果就是最終的 Prime Implicant 候選項，若無法再合併就會進入 Prime Implicant Chart。

原始組合 (來自上一次合併)	合併後位元表示 (含 -)	覆蓋的 minterms
000000-0 & 000010-0	0000-0-0	0, 2, 8, 10
00000-10 & 0000-110	0000--10	2, 6, 10, 14
00001-00 & 000011-0	00001--0	8, 10, 12, 14
000001-1 & 000011-1	0000-1-1	5, 7, 13, 15
0000011- & 0000111-	0000-11-	6, 7, 14, 15
0000110- & 0000111-	000011--	12, 13, 14, 15

```

=====
Next Column
=====
0000-0-0: 0 2 8 10
0000--10: 2 6 10 14
00001--0: 8 10 12 14
0000-1-1: 5 7 13 15
0000-11-: 6 7 14 15
000011--: 12 13 14 15
=====

```

Quine-McCluskey 的最後一步

用來找出 **必要主項** (Essential Prime Implicants) 以及所有覆蓋 minterms 的可能組合。

上方橫列：minterms (0, 2, 5, ..., 15)

左側直列：prime implicant 的二進位 (含 - 表示 don't care)

格子中 x 表示該 PI 覆蓋此 minterm

```

Prime Implicant Chart
=====
      0  2  5  6  7  8  10 12 13 14 15
-----
0000-10 | x  x      x  x  x
0000-0-0 | x  x      x  x  x
0000-1-1 | x  x      x  x  x
0000-11- | x  x      x  x  x
00001--0 | x  x      x  x  x
000011-- | x  x      x  x  x

```

若某個 minterm 只被一個 PI 覆蓋，該 PI 就是 **必要主項** (Essential Prime Implicant)

例如 minterm 0 只被 0000-0-0 覆蓋 → 這是必選主項。

Prime Implicant 對應分析

Prime Implicant (位元)	覆蓋的 minterms	Boolean 表達式 (8 變數)
0000--10	2, 6, 10, 14	$a'b'c'd'gh'$
0000-0-0	0, 2, 8, 10	$a'b'c'd'f'h'$
0000-1-1	5, 7, 13, 15	$a'b'c'd'fh$
0000-11-	6, 7, 14, 15	$a'b'c'd'fg$
00001--0	8, 10, 12, 14	$a'b'c'd'eh'$
000011--	12, 13, 14, 15	$a'b'c'd'ef$

	Prime Implicants:
0000--10	$a'b'c'd'gh'$
0000-0-0	$a'b'c'd'f'h'$
0000-1-1	$a'b'c'd'fh$
0000-11-	$a'b'c'd'fg$
00001--0	$a'b'c'd'eh'$
000011--	$a'b'c'd'ef$

必要主項 (Essential Prime Implicants)

我們觀察哪些 minterm 只被一個 PI 覆蓋，這些對應的 PI 是「必要主項」。

Minterm	唯一覆蓋它的 PI	必要主項?
0	0000-0-0	✓ 是
5	0000-1-1	✓ 是
8	0000-0-0, 00001--0	X 否
13	0000-1-1, 000011--	X 否
2	多個 PI	X 否
6	多個 PI	X 否
10	多個 PI	X 否
12	多個 PI	X 否
14	多個 PI	X 否
15	多個 PI	X 否
7	多個 PI	X 否

得出必要主項：

- ✓ 0000-0-0 → covers: 0, 2, 8, 10
- ✓ 0000-1-1 → covers: 5, 7, 13, 15

由上面兩個必要主項已涵蓋的項目為：

0, 2, 8, 10

5, 7, 13, 15

→ {0, 2, 5, 7, 8, 10, 13, 15}

剩下尚未被必要主項涵蓋的 minterms：

→ {6, 12, 14}

使用 **Petrick's Method** 補足其餘 minterms

從 Chart 中找出能覆蓋 {6, 12, 14} 的 PI：

	Prime Implicant (位元)	覆蓋的 minterms	Boolean 表達式 (8 變數)
1	0000--10	2, 6, 10, 14	a'b'c'd'gh'
2	0000-0-0	0, 2, 8, 10	a'b'c'd'fh'
3	0000-1-1	5, 7, 13, 15	a'b'c'd'fh
4	0000-11-	6, 7, 14, 15	a'b'c'd'fg
5	00001--0	8, 10, 12, 14	a'b'c'd'eh'
6	000011--	12, 13, 14, 15	a'b'c'd'ef

最小 SOP 結果

(1)6243

(2)6231

(3)5243

(4)5231

結合必要主項與上方補足項，每一組都是合法的最小 SOP：

Minimum SOPs 2進位 Chart:

```

1: 000011-- 0000-0-0 0000-11- 0000-1-1
2: 000011-- 0000-0-0 0000-1-1 0000--10
3: 00001--0 0000-0-0 0000-11- 0000-1-1
4: 00001--0 0000-0-0 0000-1-1 0000--10

```

Minimum SOPs:

```

a'b'c'd'ef a'b'c'd'f'h' a'b'c'd'fg a'b'c'd'fh
a'b'c'd'ef a'b'c'd'f'h' a'b'c'd'fh a'b'c'd'gh'
a'b'c'd'eh' a'b'c'd'f'h' a'b'c'd'fg a'b'c'd'fh
a'b'c'd'eh' a'b'c'd'f'h' a'b'c'd'fh a'b'c'd'gh'

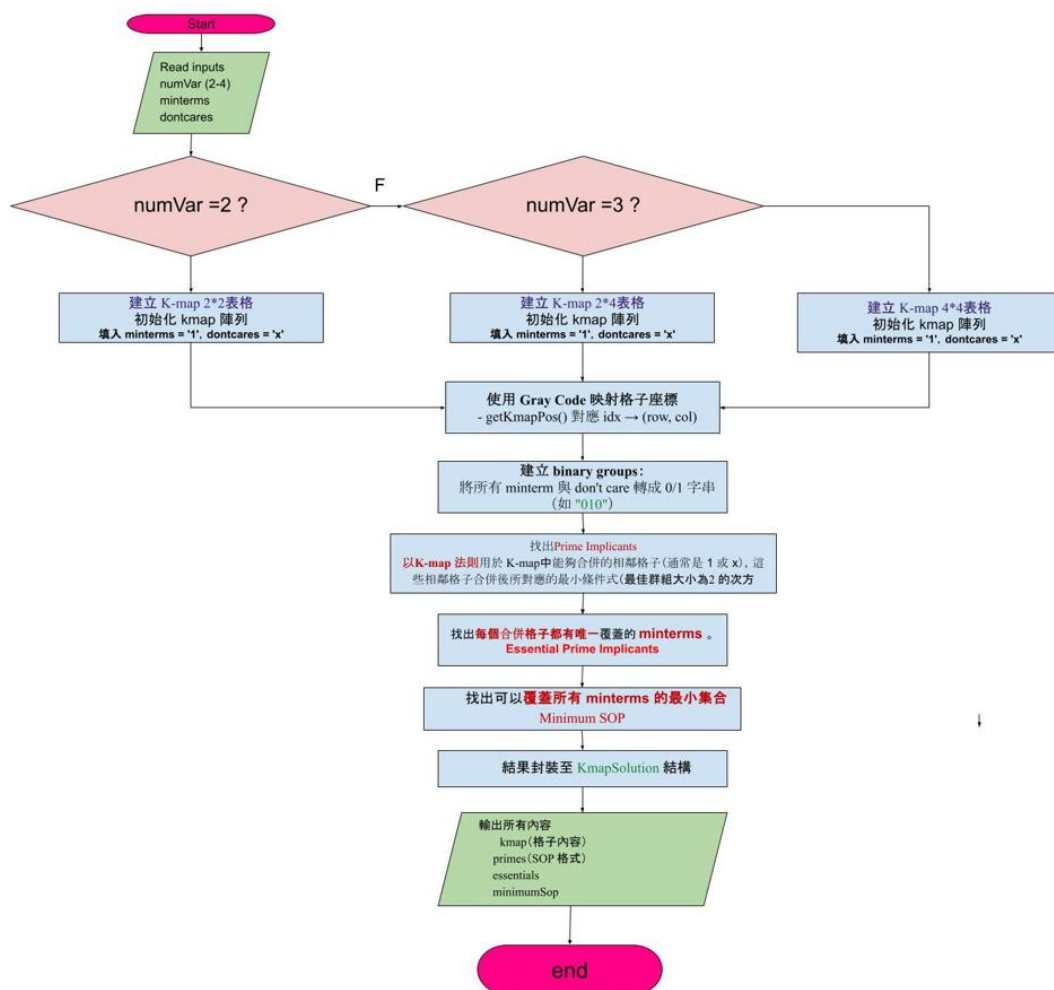
```

最小 SOP 結果表格 (以布林變數表示)

組合編號	SOP (二進位)	SOP (布林變數形式)
1	000011--	a'b'c'd'ef

	0000-0-0	a'b'c'd'f'h'
	0000-11-	a'b'c'd'fg
	0000-1-1	a'b'c'd'fh
2	000011--	a'b'c'd'ef
	0000-0-0	a'b'c'd'f'h'
	0000-1-1	a'b'c'd'fh
	0000--10	a'b'c'd'gh'
3	00001--0	a'b'c'd'eh'
	0000-0-0	a'b'c'd'f'h'
	0000-11-	a'b'c'd'fg
	0000-1-1	a'b'c'd'fh
4	00001--0	a'b'c'd'eh'
	0000-0-0	a'b'c'd'f'h'
	0000-1-1	a'b'c'd'fh
	0000--10	a'b'c'd'gh'

三、程式流程圖



上圖呈現了 K-map 求解流程圖，顯示了根據不同變數數量 (numVar) 進行 K-map 求解的步驟。下面是這張流程圖的文字描述：流程描述：

1. 開始 (Start)：

- 開始程序，讀取輸入：

- (1) numVar (變數數量，可以是 2、3 或 4)。
- (2) minterms (minterms 的索引)。
- (3) dontcares (dontcares 的索引)。

2. 分支條件：

- 根據 numVar 的值，決定如何處理：

- (1) 如果 numVar = 2，則選擇處理 2-variable K-map，並使用對應的 K-map 格式。
- (2) 如果 numVar = 3，則選擇處理 3-variable K-map，並使用對應的 K-map 格式。
- (3) 如果 numVar = 4，則選擇處理 4-variable K-map，並使用對應的 K-map 格式。

3. 填充 K-map：

- 將 minterms 和 dontcares 填充到 K-map 中，這會在 K-map 中設置 1（對應於 minterms）和 x（對應於 dontcares）。
- 4. **Gray Code 計算：**
 - 使用 Gray Code 計算格子的行和列。
 - (1) 利用 getKmapPos() 函數來確定每個格子的行（row）和列（col）位置。
- 5. **找出 Binary Groups：**
 - 將 K-map 中的相鄰格子（即相鄰的 1 或 x）進行合併，這些格子將組成新的 **binary groups**。
- 6. **找出 Prime Implicants (PI)：**
 - 根據組合後的格子，找出 K-map 中的 **Prime Implicants**，這些是合併的最小條件式。
- 7. **找出 Essential Prime Implicants (EPI)：**
 - **Essential Prime Implicants** 是唯一能夠覆蓋所有 minterms 的條件式。
如果某個 minterm 只被一個 Prime Implicant 覆蓋，那麼這個 Prime Implicant 就是 Essential Prime Implicant，因為它對應的 minterm 需要它來被覆蓋
- 8. **minimumSop：**
 - 根據 **Essential Prime Implicants** 和其他 **Prime Implicants**，進行最小化 SOP（和積型）化簡，得到最簡的邏輯表達式。
minimumSop 為可以覆蓋所有 minterms 的最小集合。
- 9. **輸出結果：**
 - 輸出 KmapSolution 結果，其中包含：
 - (1) kmap：填充過的 K-map。
 - (2) primes：所有 Prime Implicants。
 - (3) essentials：所有 Essential Prime Implicants。
 - (4) minimumSop：最小化的 SOP 表達式。
- 10. **結束 (End)：**
 - 程式執行完成，返回結果。

四、資料結構說明

1. KmapSolution 結構體（Struct KmapSolution）

成員變數名稱	資料型別	作用說明
numVar	int	儲存變數數量（2~4）

kmap	char**	儲存 K-map 表格（以字元陣列表示）
primes	vector<string>	儲存所有 prime implicants
essentials	vector<string>	儲存 essential prime implicants
minimumSop	vector<string>	儲存最小 SOP 解

2. 主程式變數說明（main.cpp）

變數名稱	資料型別	作用說明	所在位置
numVar	int	使用者輸入的變數數量（2~4）	main()
minterms	vector<int>	儲存輸入的 minterm 索引值	main()
dontcares	vector<int>	儲存輸入的 don't care 索引值	main()
sol	KmapSolution*	儲存求解後的 K-map 結果與資訊	main()

3. 核心函式說明

函式名稱	參數	回傳型別	作用說明
solveKmap	(int, vector<int>, vector<int>)	KmapSolution*	主解題函式，傳回 K-map 解與簡化結果
toBinary	(int, int)	string	將整數轉為固定長度的二進位字串
mergeBinary	(string, string)	string	合併兩個只差一位的二進位字串
binToTerm	(string)	string	將二進位轉為 SOP 表示法（如：a'b）
getKmapPos	(int, int)	pair<int, int>	將 index 轉為對應在 K-map 中的 (row, col)
findPrimeImplicants	(int, vector<int>, vector<int>)	vector<string>	找出所有 prime implicants
findEssentials	(vector<string>, vector<int>, int)	vector<string>	找出 essential prime implicants

findMinimumCover	(vector<string>, vector<int>, int)	vector<string>	找出能覆盖所有 minterm 的最小 SOP 表示
-------------------------	------------------------------------	----------------	----------------------------

五、編譯與執行結果(3 個 Case)

1. 編譯程式

編譯環境:作業系統：Windows 10 / 11

- (1) 開發工具：Microsoft Visual Studio 2022
- (2) C++ 標準：ISO C++ 20 (/std:c++20)
- (3) 編譯器：MSVC (Microsoft Visual C++)



執行流程

- 程式會要求使用者輸入：
 - 變數數量 (numVar)
 - minterms 清單
 - don't cares 清單
- 系統會自動建立並輸出：
 - K-map 表格
 - Prime implicants
 - Essential prime implicants
 - Minimum SOP 結果

執行結果說明

Case1:

```
Microsoft Visual Studio 1 x + \n\nnumVar = 2\nminterms = 0 3\ndontcares = 1\n\nnumVar = 2\nkmap = [\n  [ '1', '0' ],\n  [ 'x', '1' ]\n]\nprimes = [\"a\", \"b\"]\nessentials = [\"a\", \"b\"]\nminimumSop = [\"a\", \"b\"]
```

執行分析：

1. numVar = 2 :

K-map 有兩個變數，a 和 b，因此它是一個 2x2 的 K-map。

2. **minterms = 0, 3** :

K-map 中 0 和 3 的格子是 **minterms**，因此這些位置會被填上 1。

3. **dontcares = 1** :

這表示 **dontcare** 位置是 1，因此該位置可以任意視為 1 或 0。這樣的填充通常是用 x 表示。

4. **K-map 填充** :

根據 numVar = 2，你有一個 2x2 的 K-map :

(1) minterms 填充：對應於 minterms = {0, 3}，K-map 在位置 0 和 3 被設為 1。

(2) dontcares 填充：對應於 dontcares = {1}，K-map 在位置 1 被設為 x。

5. **Primes (Prime Implicants)** :

右圖 Prime Implicants 是指 K-map 中能夠合併的相鄰格子（通常是 1 或 x），這些相鄰格子合併後所對應的最小條件式（最佳的群組大小是 2 的次方）。這個範例中，兩個 Prime Implicants 是：

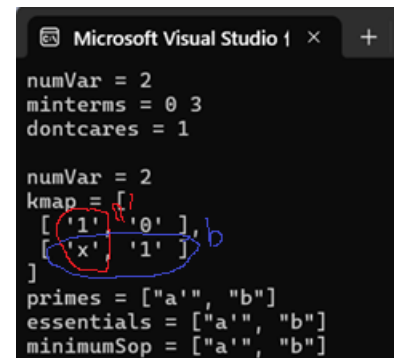
(1) a'：這個條件式覆蓋了 minterms {0} 的所有相鄰格子

(a=0, b=0) 與 (a=0, b=1) => a0 => a'

(2) b：這個條件式覆蓋了 minterms {3} 的所有相鄰格子

((a=0, b=1) 與 (a=1, b=1) => b1 => b

所以 **Primes** = ["a'", "b"]



```
Microsoft Visual Studio 1 x +
numVar = 2
minterms = 0 3
dontcares = 1

numVar = 2
kmap = [
  ['1', '0'],
  ['x', '1']
], b
]
primes = ["a'", "b"]
essentials = ["a'", "b"]
minimumSop = ["a'", "b"]
```

6. **Essentials (Essential Prime Implicants)** :

Essential Prime Implicants 是最小化邏輯式時，必須保留的 Prime Implicants。這個範例中，**由於每個格子都有唯一的** Prime Implicant，因此：
Essentials = ["a'", "b"]

7. **Minimum SOP** :

SOP (Sum of Products) 是將所有 Essential Prime Implicants 進行和積化簡。

a' + b 可以找出 **覆蓋所有 minterms {0, 3} 的最小集合**

Minimum SOP = ["a'", "b"]

Case2:

```
Microsoft Visual Studio 1 × + v
numVar = 3
minterms = 1 7
dontcares = 3 6

numVar = 3
kmap = [
  [ '0', '0', 'x', '0' ],
  [ '1', 'x', '1', '0' ]
]
primes = ["a'c", "ab", "bc"]
essentials = ["a'c"]
minimumSop = ["a'c", "ab"]
```

執行分析：

1. numVar = 3：

這表示 K-map 有三個變數 ab 和 c，因此我們將使用一個 2x4 的 K-map，總共會有 8 個格子，對應於所有 3 變數的組合。

2. minterms = 1, 7：

這代表 minterms = 1 和 7 的格子會被設為 1。這些位置對應於 K-map 中的 1。

3. dontcares = 3, 6：

這代表 dontcare = 3 和 6 的格子會被設為 x，這些位置可以任意視為 1 或 0。

4.K-map 填充：

根據 numVar = 3，我們需要使用 3 變數的 K-map，因此 K-map 的大小會是 2x4 的結構。根據 minterms 和 dontcares 來填充這個 K-map。

(1) minterms 填充：對應於 minterms = {1, 7}，K-map 在位置 1 和 7 被設為 1。

(2) dontcares 填充：對應於 dontcares = {3, 6}，K-map 在位置 3 和 6 被設為 x。

5.Prime Implicants (PI)：

右圖 Prime Implicants 是 K-map 中相鄰的格子合併後的最小條件。這個範例中，3 個 Prime Implicants 是：

(1) a'c：這個條件式覆蓋了 minterms {1}，對應於

ab=00, c=1 和 ab=01, c=1。

=>a0c1=> a'c

(2) ab：這個條件式覆蓋了 minterms {7}，對應於

ab=11, c=0 和 ab=11, c=1

=>a1b1=> ab

(3) bc：這個條件式覆蓋了 minterms {7}，對應於

ab=01, c=1 和 ab=11, c=1

=>b1c1=> bc

```
numVar = 3
minterms = 1 7
dontcares = 3 6

numVar = 3
kmap = [
  [ '0', '0', 'x', '0' ],
  [ '1', 'x', '1', '0' ]
]
primes = ["a'c", "ab", "bc"]
essentials = ["a'c"]
minimumSop = ["a'c", "ab"]
```


所以 **Primes**=["a'c","ab","bc"]

6.Essentials (Essential Prime Implicants) :

a'c : minterms {1} 是唯覆蓋 1 次的條件。

Essentials = ["a'c"]

7.Minimum SOP :

a'c+ ab or a'c+ bc 可以找出 **覆蓋所有 minterms{1,7} 的最小集合**

Minimum SOP = ["a'c ", "ab"] or Minimum SOP = ["a'c ", "bc"]

Case3:

```
Microsoft Visual Studio 1 x + v
numVar = 4
minterms = 2 5 7 11 13 14
dontcares = 3 10 15

numVar = 4
kmap = [
  [ '0', '0', '0', '0' ],
  [ '0', '1', '1', '0' ],
  [ 'x', '1', 'x', '1' ],
  [ '1', '0', '1', 'x' ]
]
primes = ["ac", "b'c", "bd", "cd"]
essentials = ["ac", "b'c", "bd"]
minimumSop = ["bd", "ac", "b'c"]
```

執行分析：

1.numVar = 4 :

這表示 K-map 有 4 個變數 ab 和 cd，因此需要使用一個 4x4 的 K-map。這會對應於所有 4 變數的組合，總共有 16 個格子。

2.minterms = {2, 5, 7, 11, 13, 14} :

這代表 minterms 在位置 2, 5, 7, 11, 13, 14 會被設為 1。

3.dontcares = {3, 10, 15} :

這代表 dontcare 位置是 3, 10, 15，這些格子會被設為 x。這些格子可以視為 1 或 0，用來進行簡化。

4.K-map 填充：

根據 numVar = 4，我們需要使用 4 變數的 K-map，因此 K-map 的大小會是 4x4。根據 minterms 和 dontcares 來填充這個 K-map。

minterms 填充：對應於 minterms = {2, 5, 7, 11, 13, 14}，K-map 在這些位置被設為 1。

dontcares 填充：對應於 dontcares = {3, 10, 15}，K-map 在這些位置被設為 x。

5.Prime Implicants (PI) :

Prime Implicants 是 K-map 中相鄰的格子合併後的最小條件。這個範例中，找到了 4 個 Prime Implicants：

(1) ac：這個條件式覆蓋了 minterms {11, 14}，對應於 K-map 中的

ab=11, ab=10 和 cd=11, cd=10。

$\Rightarrow a1c1 \rightarrow ac$

- (2) $b'c$ ：這個條件式覆蓋了 minterms {2, 11}，對應於 K-map 中

$ab=00, ab=10$ 和 $cd=11, cd=10$ 。

$\Rightarrow b0c1 \rightarrow b'c$

- (3) bd ：這個條件式覆蓋了 minterms {5, 7, 13}，對應於

$ab=01, ab=11$ 和 $cd=01, cd=11$ 。

$\Rightarrow b1d1 \rightarrow bd$

- (4) cd ：這個條件式覆蓋了 minterms {7, 11}，對應於

$ab=00, 01, 11, 10$ 和 $cd=11$

$\Rightarrow c1d1 \rightarrow cd$

所以 $\text{primes} = ["ac", "b'c", "bd", "cd"]$

6. Essentials (Essential Prime Implicants)：

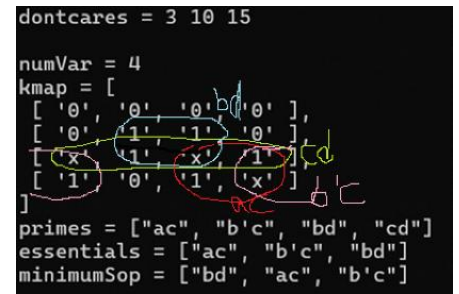
參考 Prime Implicants (PI) 發現 minterms {2, 5, 13, 14} 是唯覆蓋 1 次的條件。

所以 $\text{Essentials} = ["ac", "bd", "b'c"]$

7. Minimum SOP：

$ac + bd + b'c$ 可以找出 覆蓋所有 minterms {2, 5, 7, 11, 13, 14} 的最小集合

Minimum SOP = $["ac", "bd", "b'c"]$

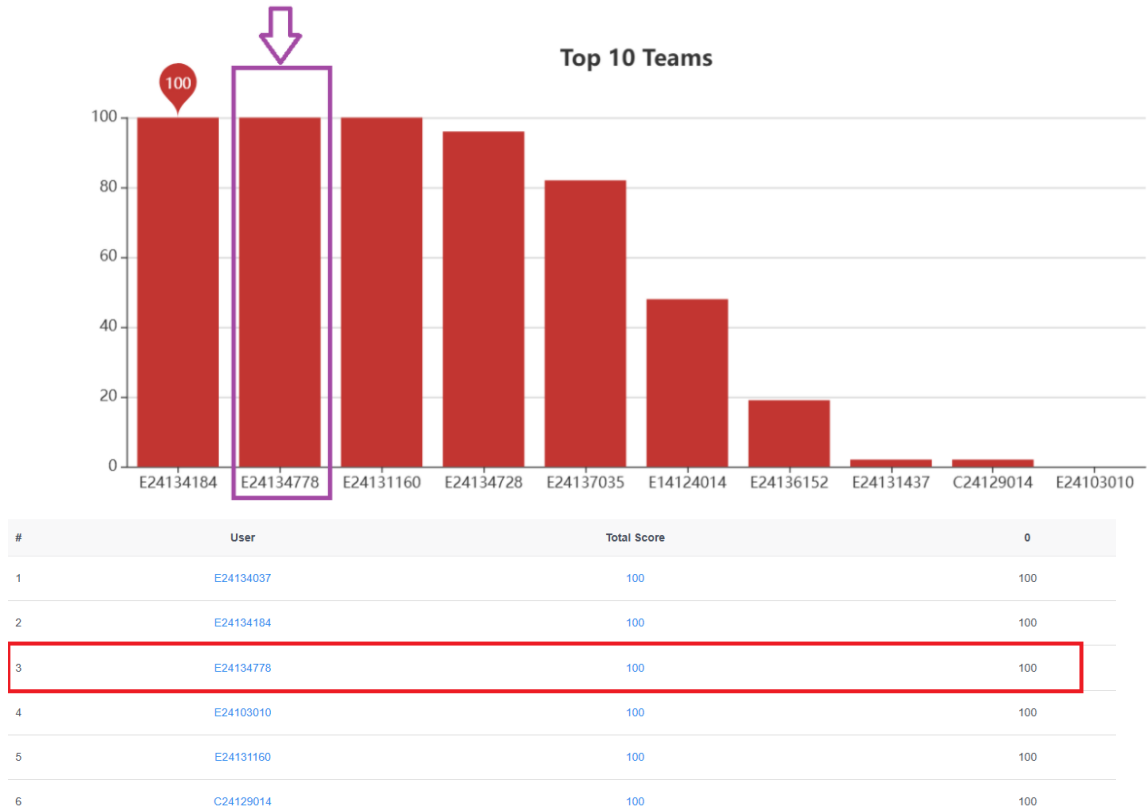


六、將程式碼上傳至 CASOJ 系統進行測試

(<https://casoj.ee.ncku.edu.tw/contest>)

測試結果如下圖 (100分)

2025 Logic System HW1 - An Auto K-map Solver



七、結論

程式成功實現了 K-map 的自動化簡化過程，能夠處理 2 到 4 變數的 K-map，並根據使用者輸入的 minterms 和 dontcares 計算 Prime Implicants、Essential Prime Implicants 以及最小化的 SOP 表達式。

這次作業讓我對 **K-map** 的簡化過程有了更深的理解，特別是在自動化簡化邏輯式、處理多變數情況下的技巧。這不僅幫助我熟悉了程式設計和數位邏輯，更讓我在面對邏輯簡化的問題時，能夠有效應用數學原則和程式邏輯。